

Lección 4: End-to-End Testing

E2E con Playwright y Mock Server

Agenda

- ¿Qué son las Pruebas E2E?
 - Playwright: E2E Moderno
 - Configuración y Variables de Entorno
 - Patrones: POM vs Tests Directos
 - Mock Server Strategy
 - Visual Testing
-

E2E: La Vista Completa

Real Browser	← Chrome, Firefox, Safari
Frontend App	← React, Vue, HTML/CSS/JS
HTTP Network	← Real API calls
Backend API	← Real server
Database	← Real data

Máxima confianza, máximo costo

¿Por qué Playwright?

Ventajas sobre Selenium:

- 🚀 3x más rápido
 - 🤖 Multi-browser automático
 - 📱 Mobile testing real
 - ⌚ Auto-wait inteligente
 - 🎥 Screenshots, videos, traces
 - 🛠 TypeScript first
-

¿Qué es CI?

CI (Continuous Integration): Integración Continua

Sistema automatizado que ejecuta tests cada vez que haces push a Git:

Developer push → GitHub Actions → Run tests →  o 

¿Cuándo se ejecuta?

- Cada push a GitHub
 - Cada Pull Request
 - Antes de hacer merge
-

Variables de Entorno (process.env) (1/2)

`process.env` : Objeto con variables de entorno del sistema

```
process.env.CI           // Variable CI (true en GitHub Actions)
process.env.NODE_ENV     // 'development' o 'production'
process.env.API_KEY      // Tu API key desde .env
```

¿De dónde vienen?

```
# .env (archivo local, NO commitear)
API_KEY=abc123
CI=true # GitHub Actions añade esto
```

Variables de Entorno (process.env) (2/2)

Uso en config:

```
retries: process.env.CI ? 2 : 1
// ¿Existe variable CI? → 2 reintentos
// ¿No existe? → 1 reintento
```

test.only y forbidOnly

test.only : Ejecuta SOLO ese test, ignora todos los demás

```
test('test 1', () => { /* este NO corre */ })
test.only('test 2', () => { /* SOLO este corre */ })
test('test 3', () => { /* este NO corre */ })
```

Problema: Si haces commit con `test.only`, CI ejecuta 1 test en vez de 100+

- ❌ Los otros 99 tests no corren
- ❌ Falsa sensación de seguridad

Solución: forbidOnly

```
forbidOnly: !!process.env.CI // En CI: FALLA si encuentra test.only
```

Configuración Real del Proyecto

```
import { defineConfig, devices } from '@playwright/test'
```

```
export default defineConfig({
  testDir: './e2e',
  forbidOnly: !!process.env.CI,
  retries: process.env.CI ? 2 : 0,

  use: {
    baseURL: 'http://localhost:5173',
    trace: 'on-first-retry',
  },

  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } }
  ],

  webServer: {
    command: 'pnpm dev',
    url: 'http://localhost:5173',
    reuseExistingServer: !process.env.CI,
  },
})
```

Ventajas: Buenos defaults, solo configuramos lo esencial

¿Qué es toHaveScreenshot? (1/2)

Visual Regression Testing (testing de regresión visual): Detecta cambios visuales no intencionales

Cómo funciona:

- 1ª ejecución: Captura "baseline" (imagen de referencia)
Guarda en: tests/__screenshots__/product-page.png
- 2ª ejecución: Captura nueva imagen
Compara pixel-por-pixel con baseline
 - ✅ PASA si son idénticas
 - ❌ FALLA si hay diferencias visuales

¿Qué es toHaveScreenshot? (2/2)

Útil para detectar:

- Cambios de colores, tamaños, posiciones
 - Regresiones CSS
 - Problemas de responsive design
 - Fuentes, espaciados, alineaciones
-

Visual Testing en Acción

```
test('Visual regression - product page', async ({ page }) => {
  await page.goto('/products/laptop')

  await expect(page).toHaveScreenshot('product-page.png')
  // 1ª vez: guarda baseline
  // Siguiendo: compara con baseline
})

test('Responsive design', async ({ page }) => {
  // Desktop
  await page.setViewportSize({ width: 1280, height: 720 })
  await expect(page).toHaveScreenshot('desktop.png')

  // Mobile
  await page.setViewportSize({ width: 375, height: 667 })
  await expect(page).toHaveScreenshot('mobile.png')
})
```

Page Object Model (POM): ¿Qué es?

Patrón de diseño que encapsula lógica de interacción con páginas en clases:

```
// Page Object: Encapsula selectores y acciones
export class ProductPage {
  constructor(private page: Page) {}

  // Selectores como getters
  get addToCartButton() {
    return this.page.getByRole('button', { name: /add to cart/i })
  }
}
```

```
// Acciones como métodos
async setQuantity(qty: number) {
  await this.page.getByRole('spinbutton').fill(qty.toString())
}

async addToCart() {
  await this.addToCartButton.click()
}
}
```

POM: Uso en Tests

```
// CON Page Object Model
test('add product with POM', async ({ page }) => {
  const productPage = new ProductPage(page)

  await page.goto('/products')
  await productPage.setQuantity(2)
  await productPage.addToCart()

  // Cambios en UI solo requieren actualizar ProductPage
  // Los tests no cambian
})
```

Ventajas:

-  Reutilización: Un Page Object para múltiples tests
 -  Mantenibilidad: Cambios de UI centralizados
 -  Legibilidad: Tests más declarativos
-

Page Object Model: Patrón de Este Proyecto

```
// CON Page Object Model (elegimos este patrón)
test('add product with POM', async ({ page }) => {
  const homePage = new HomePage(page)
  const catalogPage = new ProductCatalogPage(page)
  const cartPage = new ShoppingCartPage(page)
```

```
await homePage.open()
await catalogPage.addProductToCart(0)

await homePage.shouldHaveCartCount(1)
await cartPage.shouldHaveItems(1)
})
```

Ventajas:

-  Reutilización, Mantenibilidad
-  Legibilidad, Escalabilidad

¿Por qué Usamos POM en Este Proyecto?

Razones para usar Page Object Model:

-  Proyecto educativo: Aprender mejores prácticas
-  Escalabilidad: Preparado para crecer
-  Mantenibilidad: Cambios centralizados
-  Tests más legibles: Lenguaje del negocio

Alternativa: Tests Directos

-  Selectores duplicados en cada test
-  Difícil de mantener cuando crece

POM es la mejor práctica de la industria para E2E

User Journey Testing (Patrón Real)

```
test('Complete Shopping Journey', async ({ page }) => {
  // 1. Navegar a home
  await page.goto('/')

  // 2. Verificar productos cargados
  await expect(page.getByRole('heading', { name: /products/i })).toBeVisible

  // 3. Agregar producto al carrito
```

```
const addButton = page.getByRole('button', { name: /add to cart/i }).first
await addButton.click()

// 4. Verificar contador del carrito
await expect(page.getByTestId('cart-count')).toHaveText('1')

// 5. Ver resumen del carrito
await expect(page.getByTestId('cart-summary')).toBeVisible()
})
```

Patrón: Selectores directos + Flujo de usuario completo

Critical Business Flows

```
test('Bulk discount applies correctly', async ({ page }) => {
  await page.goto('/products')

  // Add 5+ items to trigger bulk discount
  const productCard = page.getByTestId('product-mouse')
  await productCard.getByRole('spinbutton').fill('5')
  await productCard.getByText('Add to Cart').click()

  // Verify discount
  await page.goto('/cart')
  await expect(page.getByText('Bulk Discount')).toBeVisible()
  await expect(page.getByTestId('bulk-discount'))
    .toHaveText('-$14.995')
})
```

Mock Server Strategy

```
class MockDatabase {
  private products: Product[] = [...]
  private carts = new Map<string, CartItem[]>()

  async getProducts(): Promise<Product[]> {
    await this.simulateLatency()
    return [...this.products]
  }
}
```



```
    async addToCart(sessionId: string, item: CartItem) {
      const cart = this.carts.get(sessionId) || []
      cart.push(item)
      this.carts.set(sessionId, cart)
    }
  }
}
```

In-memory database para E2E

Test Data Management

```
// Database reset para aislamiento
export class DatabaseFixture {
  async reset() {
    await fetch('/api/test/reset', { method: 'POST' })
  }

  async seedProducts() {
    await fetch('/api/test/seed/products', {
      method: 'POST',
      body: JSON.stringify({ products: [...] })
    })
  }
}

// Antes de cada test
test.beforeEach(async () => {
  await db.reset()
})
```

Error Scenarios

```
test('should handle network errors', async ({ page }) => {
  // Simulate offline
  await page.context().setOffline(true)

  await page.goto('/')
  await expect(page.getByText('Connection error')).toBeVisible()

  // Go back online
```

```
await page.context().setOffline(false)
await page.reload()
await expect(page.getByText('Our Products')).toBeVisible()
})
```

CI/CD Integration

```
# .github/workflows/e2e.yml
jobs:
  e2e-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Install dependencies
        run: npm ci
      - name: Install Playwright
        run: npx playwright install --with-deps
      - name: Run E2E tests
        run: npx playwright test
      - name: Upload artifacts
        if: failure()
        uses: actions/upload-artifact@v4
```

Mejores Prácticas

1. **Balance de Pirámide:** E2E solo para flujos críticos (5-10%)
 2. **Selectores Estables:** `getByRole` > `getByLabelText` > `getByTestId`
 3. **Esperas Inteligentes:** `waitFor`, NO `setTimeout`
 4. **Tests Independientes:** Cada test debe poder correr solo
 5. **Aislamiento de Datos:** Reset database entre tests
-

Ejercicio 1: E2E Básico con Playwright

Prompt:

Actúa como un desarrollador que escribe E2E tests con Playwright.

CONTEXT0: E2E tests validan el sistema completo: Browser + Frontend + Backend + Database. Playwright provee auto-wait inteligente (espera automáticamente a que elementos estén listos). Selectores estables: getByRole > getByLabelText > getByTestId. Los E2E tests siguen User Journeys (viajes de usuario): flujos completos que usuarios reales ejecutan

TAREA: Crea `test` E2E para agregar producto al carrito.

USER JOURNEY:

1. Navegar a home: `page.goto('/')`
2. Cambiar a vista E-commerce: click botón "E-commerce Demo"
3. Verificar productos cargaron: heading "Products" visible
4. Agregar primer producto: click botón "Add to Cart"
5. Verificar éxito: cart-summary debe ser visible

TEST REQUIREMENTS:

- Framework: Playwright (@playwright/`test`)
- Imports: `test`, `expect`
- Selectores: `getByRole` prioritario, `getByTestId` para cart-summary
- Estructura: `test.describe` + `test`
- Auto-wait: NO necesitas `waitFor()`, Playwright espera automáticamente

ARCHIVOS:

- `e2e/shopping-cart.spec.ts` (`test` E2E)

VALIDACIÓN: ejecuta `pnpm test:e2e` para verificar

Aprende: Navegación entre vistas + flujo E2E con `getByRole`

Ejercicio 2: Visual Regression

Prompt:

Actúa como un desarrollador que implementa Visual Regression Testing.

CONTEXT0: Visual Regression Testing (testing de regresión visual) detecta cambios visuales no intencionales comparando screenshots pixel-por-pixel. Primera ejecución: Playwright captura "baseline" (imagen de referencia). Sigüientes ejecuciones: Compara nueva captura con baseline. Si hay diferencia `test` FALLA y genera diff visual. `maxDiffPixels` permite tolerar pequeñas diferencias (fuentes, antialiasing).

TAREA: Crea `test` de visual regression para carrito con `item`.

TEST FLOW:

1. Navegar a home: `page.goto('/')`
2. Agregar primer producto al carrito (click "Add to Cart")
3. Esperar que cart-count muestre "1" (sincronización)
4. Capturar screenshot: `expect(page).toHaveScreenshot('cart-with-item.png')`
5. Configurar tolerancia: `maxDiffPixels: 100`

TEST REQUIREMENTS:

- Framework: Playwright (@playwright/test)
- Assertion: `toHaveScreenshot()` con opciones
- Tolerancia: `maxDiffPixels: 100` (permite hasta 100 pixels diferentes)
- Sincronización: esperar cart-count antes del screenshot

PRIMERA EJECUCIÓN:

- Playwright guardará baseline en: `e2e/__screenshots__/cart-with-item.png`
- Test PASARÁ (no hay baseline aún para comparar)

SIGUIENTES EJECUCIONES:

- Compara nuevo screenshot con baseline
- Si hay cambios CSS/layout, `test` FALLA y muestra diff visual

ARCHIVOS:

- `e2e/visual-regression.spec.ts` (`test` visual)

VALIDACIÓN: ejecuta `pnpm test:e2e --update-snapshots` (primera vez)

Aprende: Visual regression testing automático con Playwright

Puntos Clave

1. **Escenarios Reales:** Testa lo que usuarios hacen
2. **Sistema Completo:** Backend + Frontend + Database
3. **Crítico para el Negocio:** Protege flujos de revenue
4. **Quality Gate** (compuerta de calidad): Previene regresiones mayores
5. **Playwright:** Herramienta moderna y poderosa
6. **Mock Server:** Control total del entorno de testing